

Project1

可扩展哈希

Directory: 是存放bucket指针的容器, 可动态生长 (以原大小的倍数作为增长率), 容器的每个元素可用哈希值来索引。

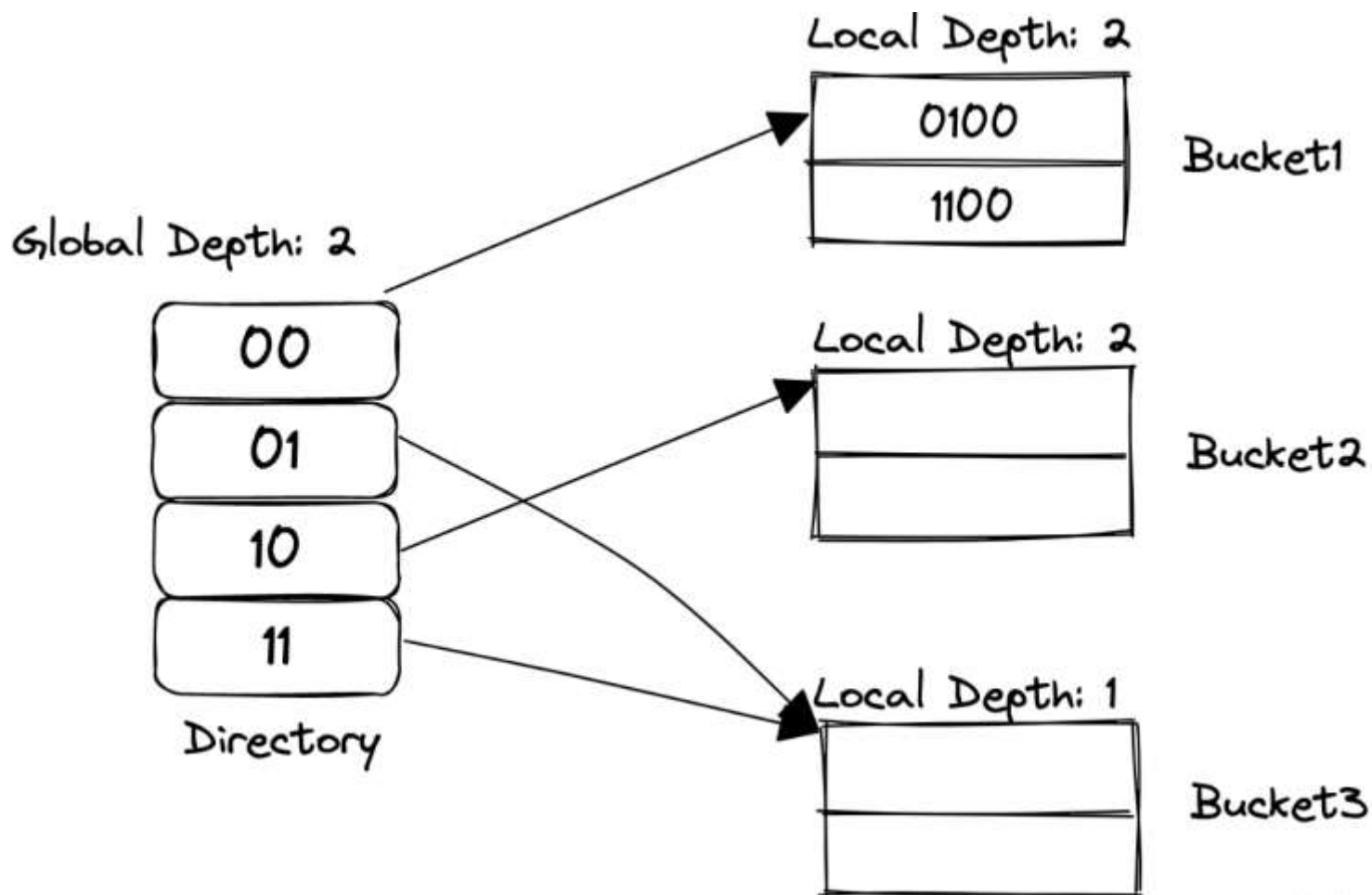
Bucket: 桶。存放Key/value pair的桶, 数据结构层面是一个线性表。

Global Depth: 假设global depth为 n , 那么当前的directory必定有 2^n 个条目。同时决定使用哈希值的低 n 位来计算目录索引

Local Depth: local depth指的是 (假设local depth为 n), 在当前的bucket之下, 每个元素的key的低 n 位都是相同的。

计算当前bucket有几个指针指向它的公式是:

$$2^{\text{globalDepth} - \text{localDepth}}$$



查找/删除

1. 首先使用哈希函数对键进行哈希
2. 根据全局深度确定需要查看哈希值的前几位
3. 通过这几位确定目录中的索引位置
4. 找到对应的桶
5. 在该桶内线性搜索查找具体的键值对
6. 如果找到，就从桶中移除该键值对（删除）

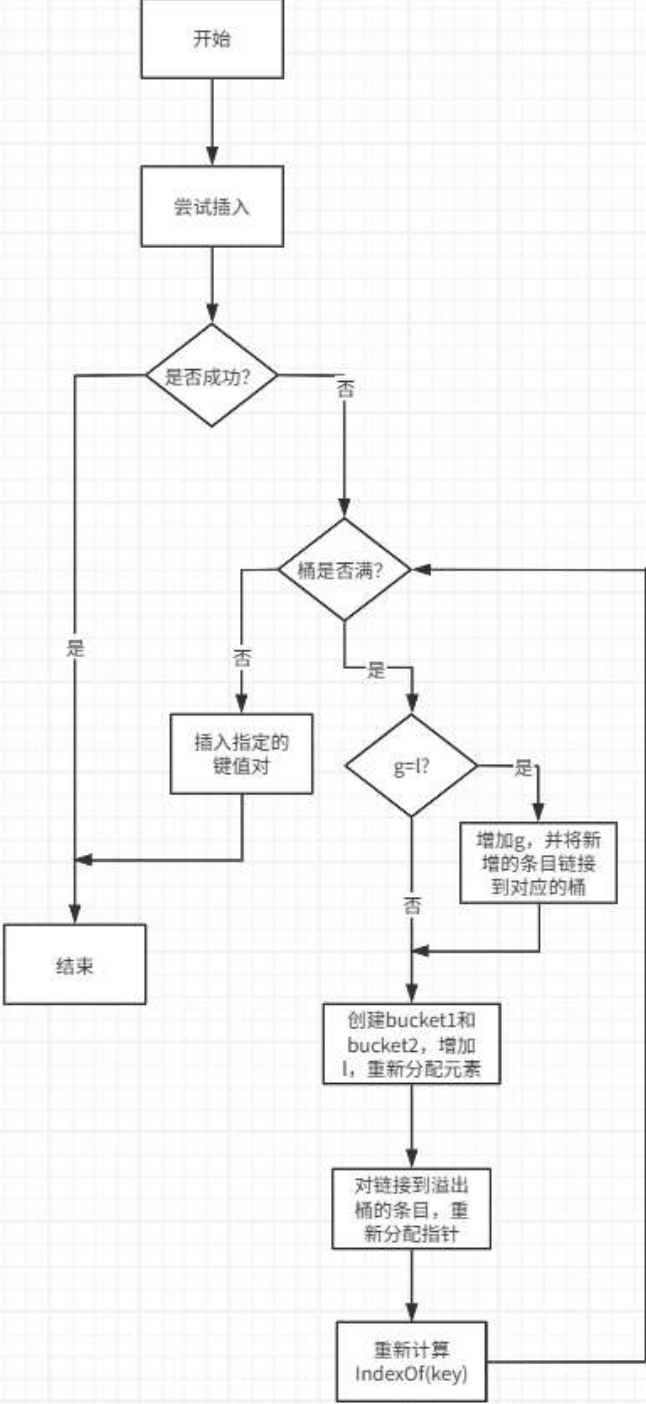
```
1 template <typename K, typename V>
2 auto ExtendibleHashTable<K, V>::Find(const K &key, V &value) -> bool {
3     std::scoped_lock<std::mutex> lock(latch_);
4     auto index = IndexOf(key);
5     auto bucket = dir_[index];
6     return bucket->Find(key, value);
7 }
```

```
1 template <typename K, typename V>
2 auto ExtendibleHashTable<K, V>::Bucket::Find(const K &key, V &value) -> bool {
3     for(const auto &[key_, value_] : list_) {
4         if(key_ == key) {
5             value = value_;
6             return true;
7         }
8     }
9     return false;
10 }
```

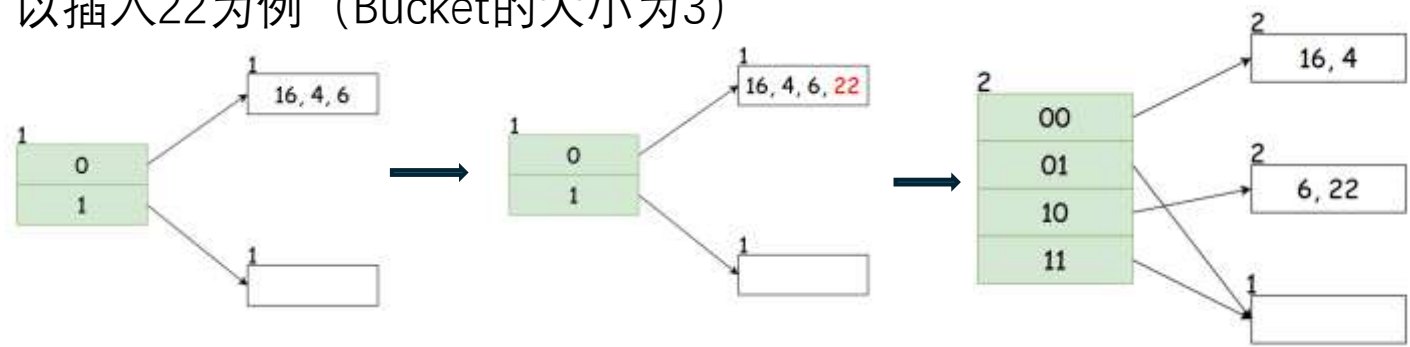
```
1 template <typename K, typename V>
2 auto ExtendibleHashTable<K, V>::Remove(const K &key) -> bool {
3     std::scoped_lock<std::mutex> lock(latch_);
4     auto index = IndexOf(key);
5     auto bucket = dir_[index];
6     return bucket->Remove(key);
7 }
```

```
1 template <typename K, typename V>
2 auto ExtendibleHashTable<K, V>::Bucket::Remove(const K &key) -> bool {
3     for(auto it = list_.begin(); it != list_.end(); it++) {
4         if(it->first == key) {
5             list_.erase(it);
6             return true;
7         }
8     }
9     return false;
10 }
```

插入



以插入22为例 (Bucket的大小为3)



1. 当插入元素 22 之后, Bucket的大小为4, 超过Bucket的大小限制,尝试插入失败。因为该Bucket已满, 则Bucket分裂。分裂之后, 两个Bucket的Local Depth为2, 大于Global Depth, Global Depth也需要加一, 变为2, 然后目录Directory 扩充, 变为原来的两倍。
2. 由于分裂的Bucket的Local Depth为2, 所以使用Index的两位作为索引, 因此00和10分别指向分裂的两个Bucket, 分裂之前的Bucket的Index为0, 新增一位后就是 00 和 10. 原来Index为 1的Bucket没有分裂, 它的Local Depth为1, 所以 01 和 11 的Index均指向该Bucket.

```

1  template <typename K, typename V>
2  void ExtendibleHashTable<K, V>::Insert(const K &key, const V &value) {
3      std::scoped_lock<std::mutex> lock(latch_);
4      while(dir_[IndexOf(key)]->IsFull()) {
5          auto index = IndexOf(key);
6          auto bucket = dir_[index];
7
8          if(bucket->GetDepth() == GetGlobalDepthInternal()) {
9              global_depth++;
10             size_t oldSize = dir_.size();
11             dir_.resize(oldSize << 1);
12             for(size_t i = 0; i < oldSize; i++) {
13                 dir_[i + oldSize] = dir_[i];
14             }
15         }
16         int bit = 1 << bucket->GetDepth();
17         auto bucket0 = std::make_shared<Bucket>(bucket_size_, bucket->GetDepth() + 1);
18         auto bucket1 = std::make_shared<Bucket>(bucket_size_, bucket->GetDepth() + 1);
19         for(const auto &[key_, value_] : bucket->GetItems()) {
20             size_t hash_key = std::hash<K>()(key_);
21             if(hash_key & bit) {
22                 bucket1->Insert(key_, value_);
23             }
24             else {
25                 bucket0->Insert(key_, value_);
26             }
27         }
28         num_buckets++;
29         for(size_t i = 0; i < dir_.size(); i++) {
30             if(dir_[i] == bucket) {
31                 dir_[i] = (i & bit) ? bucket1 : bucket0;
32             }
33         }
34     }
35     auto index = IndexOf(key);
36     auto bucket = dir_[index];
37     for(auto &[key_, value_] : bucket->GetItems()) {
38         if(key_ == key) {
39             value_ = value;
40             return;
41         }
42     }
43     bucket->Insert(key, value);
44 }

```

```

1  template <typename K, typename V>
2  auto ExtendibleHashTable<K, V>::Bucket::Insert(const K &key, const V &value) -> bool {
3      if(list_.size() < size_) {
4          list_.push_back(std::make_pair(key, value));
5          return true;
6      }
7      return false;
8  }

```

LRU-K

淘汰

1. 优先淘汰**未成熟队列**中的帧（访问次数 $< k$ ）
2. 如果未成熟队列为空，则淘汰**成熟队列**中的帧
3. 由于std::set自动排序，队列第一个元素就是最早访问的帧

对于同一个帧，它在两个队列中，只会出现一次（当次数 $< k$ 时，在lru_premature，次数 $> k$ 时，在mature中）

因为一个它的位置是确定且唯一的，所以我的顺序不影响驱逐（无论我先查找哪个队列，如果它存在，一定能找到）。

```
1 auto LRUReplacer::Evict(frame_id_t *frame_id) -> bool {
2     std::scoped_lock<std::mutex> lock(latch_);
3     if (lru_premature_.empty() && lru_mature_.empty()) {
4         return false;
5     }
6     auto has_premature = !lru_premature_.empty();
7     auto first_iter = has_premature ? lru_premature_.begin() : lru_mature_.begin();
8     *frame_id = (*first_iter)->GetFrameId();
9     DeallocateFrameRecord(first_iter, has_premature);
10    return true;
11 }
```


记录访问

1. 新帧：创建记录但不加入队列
(默认不可驱逐)
2. 未成熟帧：访问次数 $< k$ ，保持在未成熟队列
3. 关键状态：当访问次数从 $k-1 \rightarrow k$ 时，从未成熟队列移动到成熟队列
4. 成熟帧：每次访问后先移除再重新插入，以更新排序位置

```
1 void LRUKReplacer::RecordAccess(frame_id_t frame_id) {
2     std::scoped_lock<std::mutex> lock(latch_);
3     if (frames_[frame_id] == nullptr) {
4         AllocateFrameRecord(frame_id);
5     }
6     auto is_permature = frames_[frame_id]->AccessSize() < k_;
7     auto is_evictable = frames_[frame_id]->IsEvictable();
8     // 已经是k-1次访问，这次访问后需要加入到mature中，所以先在permature中remove
9     // 没达到k-1次访问的话，还是在permature中，记录新的访问时间
10    if (is_evictable && is_permature && frames_[frame_id]->AccessSize() == (k_ - 1)) {
11        // from premature to mature
12        lru_permature_.erase(frames_[frame_id]);
13    }
14    // 已经在mature中，先删除在插入，lru
15    if (is_evictable && (!is_permature)) {
16        lru_mature_.erase(frames_[frame_id]);
17    }
18    frames_[frame_id]->Access(CurrTime());
19
20    if (is_evictable && is_permature && frames_[frame_id]->AccessSize() == k_) {
21        lru_mature_.insert(frames_[frame_id]);
22    }
23
24    if (is_evictable && (!is_permature)) {
25        lru_mature_.insert(frames_[frame_id]);
26    }
27 }
```

可驱逐状态管理

1. 只有可驱逐的帧才会被放入队列
2. 状态改变时更新replacer_size_计数
3. 根据访问次数决定放入哪个队列

```
1 void LRUKReplacer::SetEvictable(frame_id_t frame_id, bool set_evictable) {
2     std::scoped_lock<std::mutex> lock(latch_);
3     if (frames_[frame_id] == nullptr) {
4         return;
5     }
6     auto is_permature = frames_[frame_id]->AccessSize() < k_;
7     if (set_evictable && !frames_[frame_id]->IsEvictable()) {
8         // not evictable -> evictable
9         replacer_size_++;
10        if (is_permature) {
11            lru_permature_.insert(frames_[frame_id]);
12        } else {
13            lru_mature_.insert(frames_[frame_id]);
14        }
15    }
16
17    if (!set_evictable && frames_[frame_id]->IsEvictable()) {
18        // evictable -> not evictable
19        replacer_size_--;
20        if (is_permature) {
21            lru_permature_.erase(frames_[frame_id]);
22        } else {
23            lru_mature_.erase(frames_[frame_id]);
24        }
25    }
26
27    frames_[frame_id]->SetEvictable(set_evictable);
28 }
```

缓冲池管理

创建新页面

1.查找可用帧：调用FindVictim()获取空闲帧或驱逐一个帧

2.分配页面ID：使用AllocatePage()生成新的页面ID

3.重置页面元数据：

1. 清空页面数据 (ResetMemory)
2. 设置新的page_id
3. pin_count设置为1 (新页面被固定)
4. is_dirty设置为false

4.更新替换器：

1. RecordAccess(): 记录访问
2. SetEvictable(false): 标记为不可驱逐 (因为被固定)

5.更新页表：插入page_id到frame_id的映射

6.返回页面指针

```
1 auto BufferPoolManagerInstance::NewPgImp(page_id_t *page_id) -> Page * {
2     std::scoped_lock<std::mutex> lock(latch_);
3     // first check if there is any available frame
4     frame_id_t available_frame_id = -1;
5     if (!FindVictim(&available_frame_id)) {
6         return nullptr;
7     }
8     // at this point , we know there is indeed available frame
9     page_id_t new_page_id = AllocatePage();
10    // reset memory and mata for new page
11    pages_[available_frame_id].ResetMemory();
12    pages_[available_frame_id].page_id_ = new_page_id;
13    pages_[available_frame_id].pin_count_ = 1;
14    pages_[available_frame_id].is_dirty_ = false;
15    // let replacer track this new page
16    replacer_>RecordAccess(available_frame_id);
17    replacer_>SetEvictable(available_frame_id, false);
18    // maping pageid
19    page_table_>Insert(new_page_id, available_frame_id);
20    *page_id = new_page_id;
21    return &pages_[available_frame_id];
22 }
```


获取现有页面

1.检查页面是否在缓冲池:

1. 通过page_table_->Find()查找
2. 如果在, 更新替换器记录, 增加pin_count, 返回页面

2.页面不在缓冲池:

1. 调用FindVictim()获取可用帧
2. 重置页面元数据
3. **关键:** 从磁盘读取页面数据 disk_manager_->ReadPage()

3.更新替换器和页表

4.返回页面指针

```
1 auto BufferPoolManagerInstance::FetchPgImp(page_id_t page_id) -> Page * {
2     std::scoped_lock<std::mutex> lock(latch_);
3     // first search the page_id in buffer pool
4     frame_id_t existing_frame_id = -1;
5     if (page_table_->Find(page_id, existing_frame_id)) {
6         // this page in buffer pool
7         replacer_->RecordAccess(existing_frame_id);
8         replacer_->SetEvictable(existing_frame_id, false);
9         pages_[existing_frame_id].pin_count++;
10        return &pages_[existing_frame_id];
11    }
12    // not currently in buffer pool, need first try to evict a frame out, then read this page from disk to memory
13    frame_id_t available_frame_id = -1;
14    if (!FindVictim(&available_frame_id)) {
15        // no evictable frame
16        return nullptr;
17    }
18    pages_[available_frame_id].ResetMemory();
19    pages_[available_frame_id].page_id = page_id;
20    pages_[available_frame_id].pin_count = 1;
21    pages_[available_frame_id].is_dirty_ = false;
22    // fetch the required page's actual data from disk
23    disk_manager_->ReadPage(page_id, pages_[available_frame_id].GetData());
24
25    replacer_->RecordAccess(available_frame_id);
26    replacer_->SetEvictable(available_frame_id, false);
27    page_table_->Insert(page_id, available_frame_id);
28    return &pages_[available_frame_id];
29 }
```

释放页面

1.检查页面是否存在：通过页表查找frame_id

2.检查pin_count：如果已经是0，返回false

3.减少pin_count：如果为0，变为可驱逐

4.更新脏标志

```
1  auto BufferPoolManagerInstance::UnpinPgImp(page_id_t page_id, bool is_dirty) -> bool {
2      std::scoped_lock<std::mutex> lock(latch_);
3      frame_id_t frame_id = -1;
4      if (!page_table_->Find(page_id, frame_id)) {
5          // page id page is not in buffer pool
6          return false;
7      }
8      if (pages_[frame_id].GetPinCount() == 0) {
9          return false;
10     }
11     if (--pages_[frame_id].pin_count_ == 0) {
12         // pin count reaches 0, become evictable
13         replacer_->SetEvictable(frame_id, true);
14     }
15     if (is_dirty) {
16         pages_[frame_id].is_dirty_ = is_dirty;
17     }
18     return true;
19 }
```

删除页面

- 1.检查页面是否存在：不在缓冲池则直接返回true
- 2.检查是否被固定：pin_count > 0则不能删除，返回false
- 3.处理脏页：如果是脏页，写回磁盘
- 4.清理资源：
 1. 从页表中移除映射
 2. 从替换器中移除记录
 3. 将帧加入空闲列表
 4. 重置页面元数据
- 5.释放磁盘空间

```
1 auto BufferPoolManagerInstance::DeletePgImp(page_id_t page_id) -> bool {
2     std::scoped_lock<std::mutex> lock(latch_);
3     frame_id_t frame_id = -1;
4     if (!page_table_->Find(page_id, frame_id)) {
5         // page id page is not in buffer pool
6         return true;
7     }
8     if (pages_[frame_id].pin_count_ != 0) {
9         // currently pinned cannot be deleted
10        return false;
11    }
12    // if dirty , write back to disk
13    if (pages_[frame_id].IsDirty()) {
14        disk_manager_->WritePage(pages_[frame_id].GetPageId(), pages_[frame_id].GetData());
15        pages_[frame_id].is_dirty_ = false;
16    }
17    page_table_->Remove(page_id);
18    replacer_->Remove(frame_id);
19    free_list_.push_back(frame_id);
20    // reset page metadata
21    pages_[frame_id].ResetMemory();
22    pages_[frame_id].ResetMemory();
23    pages_[frame_id].ResetMemory();
24    pages_[frame_id].is_dirty_ = false;
25    // imitate free the page on disk
26    DeallocatePage(page_id);
27    return true;
28 }
```



```

panan@lancher:~/Desktop/bustub_initial/build$ ./test/extendible_hash_table_test
Running main() from gmock_main.cc
[=====] Running 12 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 12 tests from ExtendibleHashTableTest
[ RUN      ] ExtendibleHashTableTest.InsertSplit
[ OK       ] ExtendibleHashTableTest.InsertSplit (1 ms)
[ RUN      ] ExtendibleHashTableTest.InsertMultipleSplit
[ OK       ] ExtendibleHashTableTest.InsertMultipleSplit (1 ms)
[ RUN      ] ExtendibleHashTableTest.ConcurrentInsertFind
[ OK       ] ExtendibleHashTableTest.ConcurrentInsertFind (141 ms)
[ RUN      ] ExtendibleHashTableTest.ConcurrentRemoveInsert
[ OK       ] ExtendibleHashTableTest.ConcurrentRemoveInsert (167 ms)
[ RUN      ] ExtendibleHashTableTest.InitiallyEmpty
[ OK       ] ExtendibleHashTableTest.InitiallyEmpty (0 ms)
[ RUN      ] ExtendibleHashTableTest.InsertAndFind
[ OK       ] ExtendibleHashTableTest.InsertAndFind (1 ms)
[ RUN      ] ExtendibleHashTableTest.GlobalDepth
[ OK       ] ExtendibleHashTableTest.GlobalDepth (0 ms)
[ RUN      ] ExtendibleHashTableTest.LocalDepth
[ OK       ] ExtendibleHashTableTest.LocalDepth (0 ms)
[ RUN      ] ExtendibleHashTableTest.InsertAndReplace
[ OK       ] ExtendibleHashTableTest.InsertAndReplace (0 ms)
[ RUN      ] ExtendibleHashTableTest.Remove
[ OK       ] ExtendibleHashTableTest.Remove (1 ms)
[ RUN      ] ExtendibleHashTableTest.GetNumBuckets
[ OK       ] ExtendibleHashTableTest.GetNumBuckets (0 ms)
[ RUN      ] ExtendibleHashTableTest.IntegratedTest
[ OK       ] ExtendibleHashTableTest.IntegratedTest (21 ms)
[-----] 12 tests from ExtendibleHashTableTest (339 ms total)

[-----] Global test environment tear-down
[=====] 12 tests from 1 test suite ran. (339 ms total)
[ PASSED ] 12 tests.

```

```

panan@lancher:~/Desktop/bustub_initial/build$ ./test/buffer_pool_manager_instance_test
Running main() from gmock_main.cc
[=====] Running 13 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 13 tests from BufferPoolManagerInstanceTest
[ RUN      ] BufferPoolManagerInstanceTest.SampleTest
[ OK       ] BufferPoolManagerInstanceTest.SampleTest (2 ms)
[ RUN      ] BufferPoolManagerInstanceTest.BinaryDataTest
[ OK       ] BufferPoolManagerInstanceTest.BinaryDataTest (1 ms)
[ RUN      ] BufferPoolManagerInstanceTest.NewPage
[ OK       ] BufferPoolManagerInstanceTest.NewPage (1 ms)
[ RUN      ] BufferPoolManagerInstanceTest.UnpinPage
[ OK       ] BufferPoolManagerInstanceTest.UnpinPage (1 ms)
[ RUN      ] BufferPoolManagerInstanceTest.FetchPage
[ OK       ] BufferPoolManagerInstanceTest.FetchPage (5 ms)
[ RUN      ] BufferPoolManagerInstanceTest.DeletePage
[ OK       ] BufferPoolManagerInstanceTest.DeletePage (3 ms)
[ RUN      ] BufferPoolManagerInstanceTest.IsDirty
[ OK       ] BufferPoolManagerInstanceTest.IsDirty (1 ms)
[ RUN      ] BufferPoolManagerInstanceTest.ConcurrencyTest
[ OK       ] BufferPoolManagerInstanceTest.ConcurrencyTest (359 ms)
[ RUN      ] BufferPoolManagerInstanceTest.IntegratedTest
[ OK       ] BufferPoolManagerInstanceTest.IntegratedTest (902 ms)
[ RUN      ] BufferPoolManagerInstanceTest.HardTest_1
[ OK       ] BufferPoolManagerInstanceTest.HardTest_1 (1157 ms)
[ RUN      ] BufferPoolManagerInstanceTest.HardTest_2
[ OK       ] BufferPoolManagerInstanceTest.HardTest_2 (941 ms)
[ RUN      ] BufferPoolManagerInstanceTest.HardTest_3
[ OK       ] BufferPoolManagerInstanceTest.HardTest_3 (1642 ms)
[ RUN      ] BufferPoolManagerInstanceTest.HardTest_4
[ OK       ] BufferPoolManagerInstanceTest.HardTest_4 (3942 ms)
[-----] 13 tests from BufferPoolManagerInstanceTest (8965 ms total)

[-----] Global test environment tear-down
[=====] 13 tests from 1 test suite ran. (8965 ms total)
[ PASSED ] 13 tests.

```

```

panan@lancher:~/Desktop/bustub_initial/build$ ./test/lru_k_replacer_test
Running main() from gmock_main.cc
[=====] Running 4 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 4 tests from LRUKReplacerTest
[ RUN      ] LRUKReplacerTest.SampleTest
[ OK       ] LRUKReplacerTest.SampleTest (1 ms)
[ RUN      ] LRUKReplacerTest.Evict
[ OK       ] LRUKReplacerTest.Evict (115 ms)
[ RUN      ] LRUKReplacerTest.Size
[ OK       ] LRUKReplacerTest.Size (1 ms)
[ RUN      ] LRUKReplacerTest.ConcurrencyTest
[ OK       ] LRUKReplacerTest.ConcurrencyTest (77 ms)
[-----] 4 tests from LRUKReplacerTest (196 ms total)

[-----] Global test environment tear-down
[=====] 4 tests from 1 test suite ran. (197 ms total)
[ PASSED ] 4 tests.

```


Project2

单线程 B+树和多线程B+树

单线程B+树

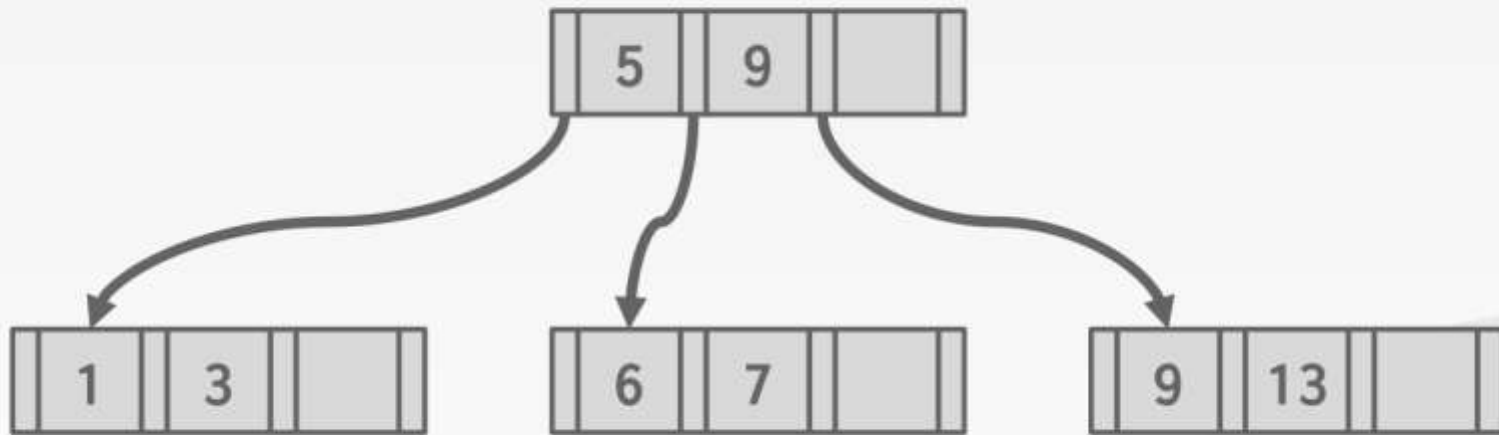
1. B+ 树节点 —— 页 (Page)

- 在 Bustub 中, **B+ 树的每个节点都是一个 Page**
- Page 大小固定为 **4KB**
- 通过 Buffer Pool Manager 管理
- Internal Page 和 Leaf Page **不同**
- 页内数据=header+键值对



Internal Page

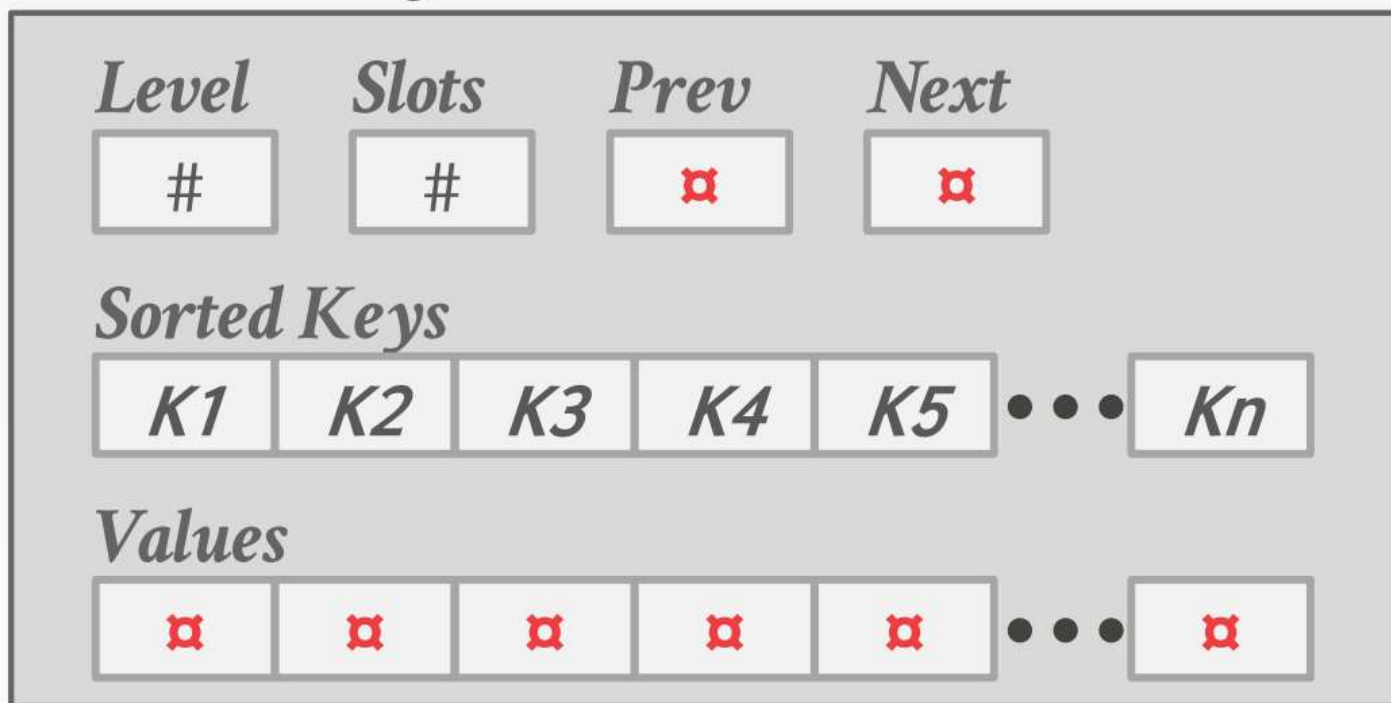
- N阶B+树的内部节点共N个键值，将键值分为N+1个区间
- N+1个区间对应N+1个儿子节点



Leaf Page

- N阶B+树的叶子节点共N个键值，对应N个KV对

B+Tree Leaf Node

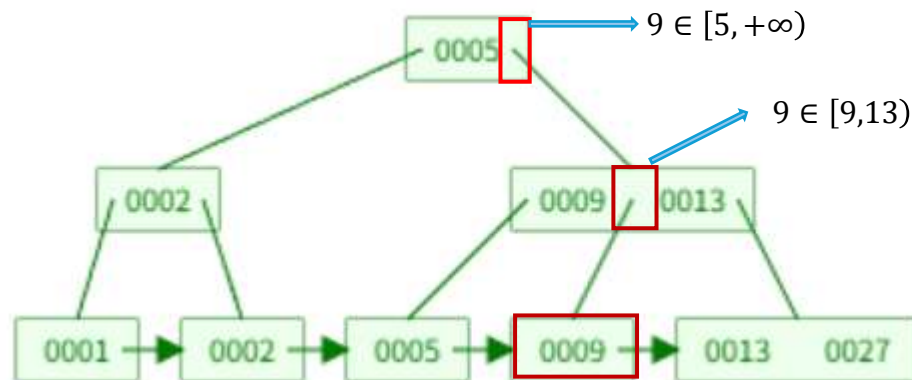


2.B+树的结构特点

- 完全平衡树-所有叶子节点的高度相同
- 所有数据只存储在叶子节点
- 内部节点仅作为索引
- 叶子节点通过链表连接-可以顺序访问

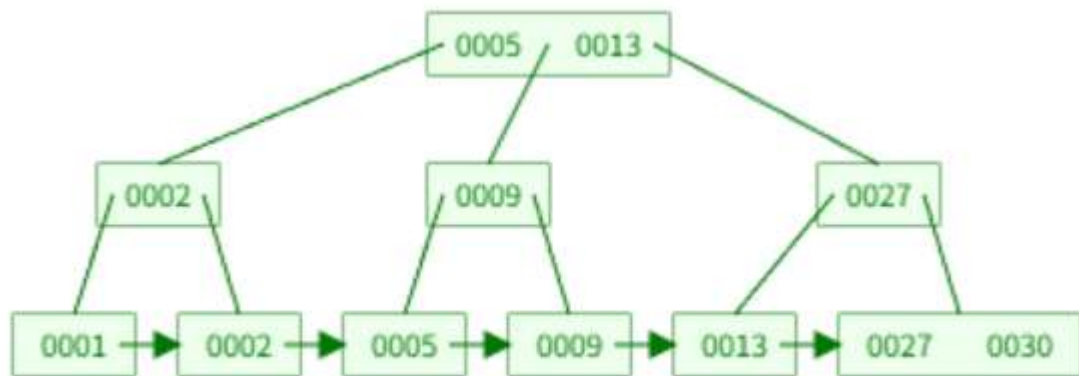
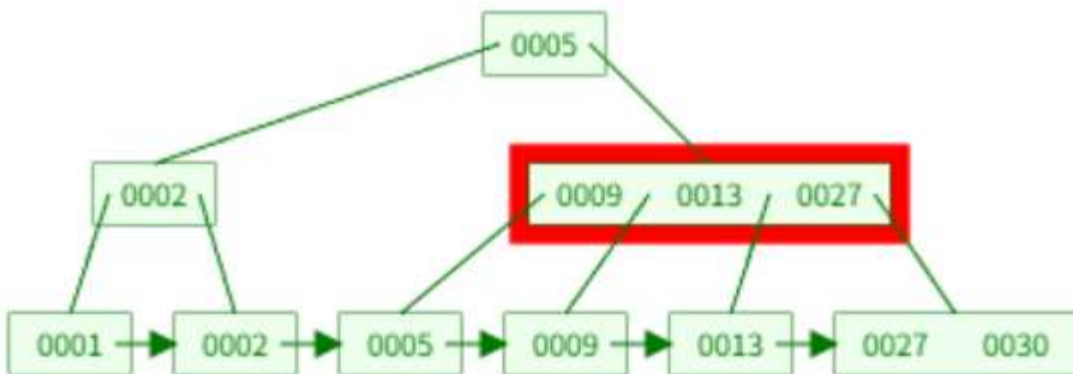
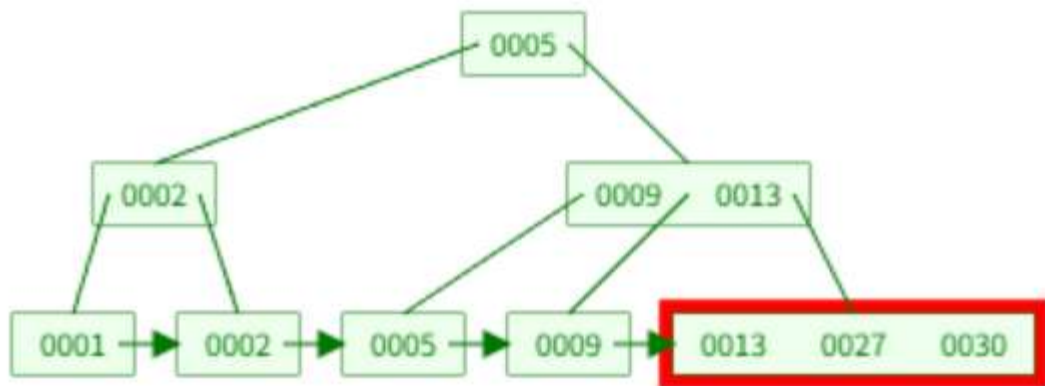
3.B+树的查询

- 以查找9为例
- 1.首先从根节点出发，如果是内部节点，就二分找到第一个键值大于等于所需键值9的儿子节点
- 2.如果是内部节点，继续重复步骤1
- 如果是叶子节点，二分查找第一个键值大于等于所需键值9的KV对，如果找到的键值不等于所需键值9，则返回false。否则返回true。



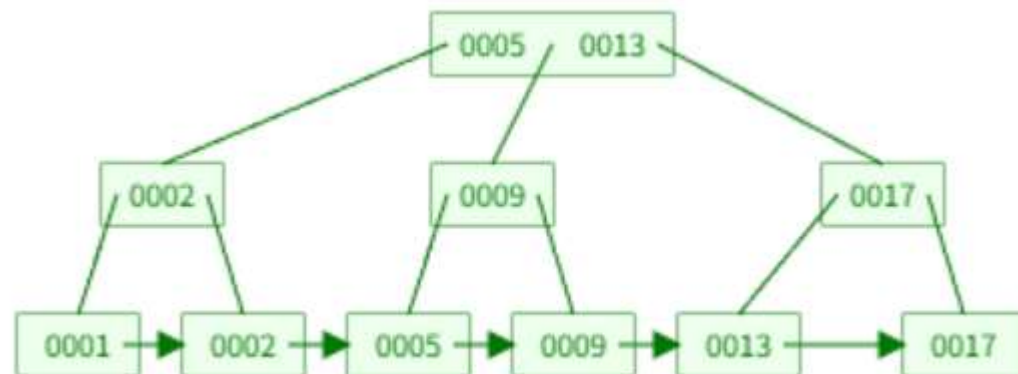
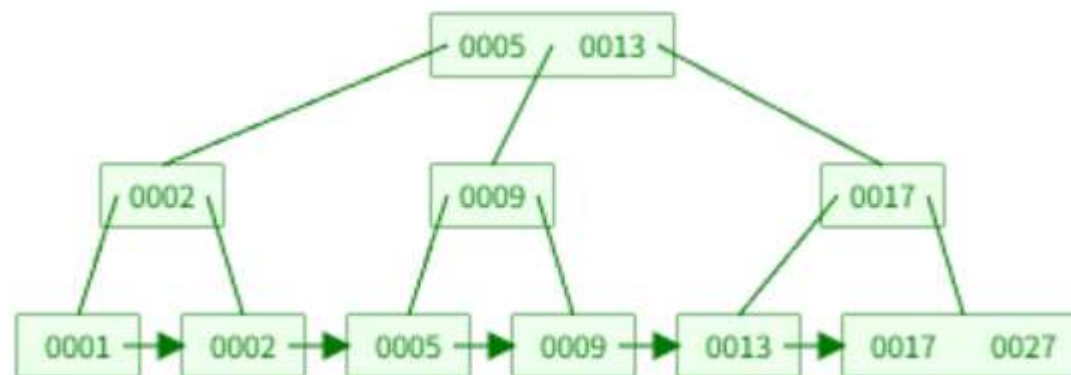
4.B+树的插入

- 以插入30为例
- 根据查询逻辑，找到叶子节点中包含所需键值的节点，然后二分找到需要插入的位置，调用插入操作。
- 如果该节点满，则需要进行分裂操作。如果未满足则直接插入后结束。
- 1.叶子节点的分裂：将叶子节点分为两半，选择右边的第一个键值作为新插入的键值插入到父亲节点中。
- 2.内部节点的分裂：将内部节点分成两半，同样选择右边节点的第一个键值作为新插入的键值插入到父亲节点中。此时不同的是分类后的另一半的第一个键值不在右边节点中，而是成为父亲节点新插入的键



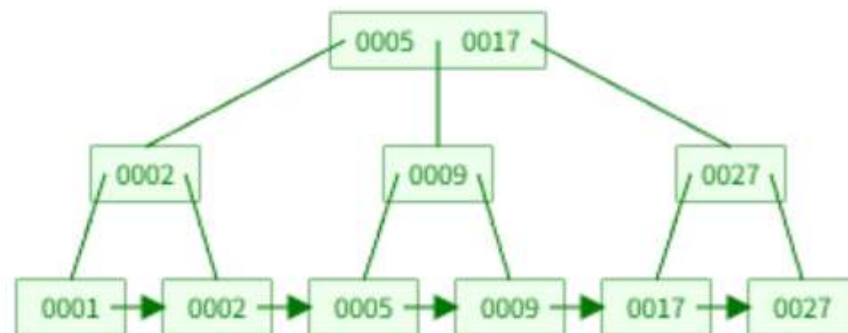
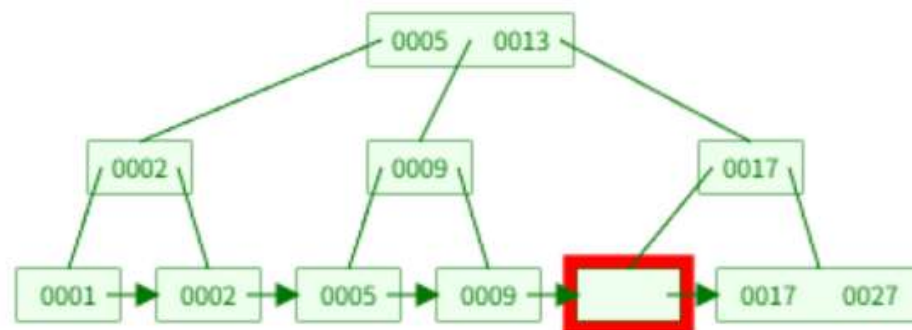
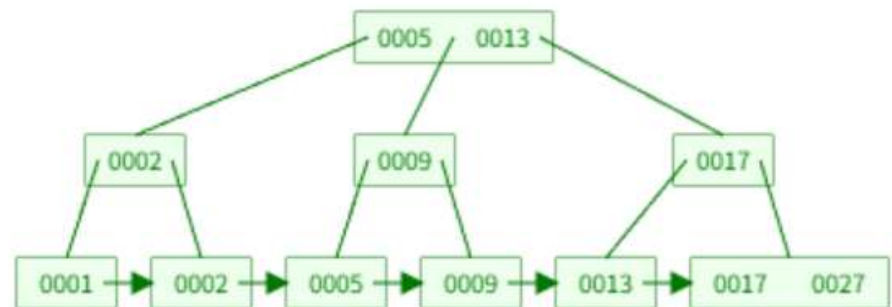
5.B+树的删除

- **情况一：可以直接删除**
- 1. 找到包含要删除键的节点。
- 2. 从节点中删除该键。
- 3. 删除后，节点的键的数量仍然大于最小允许数量，可直接删除后保持平衡



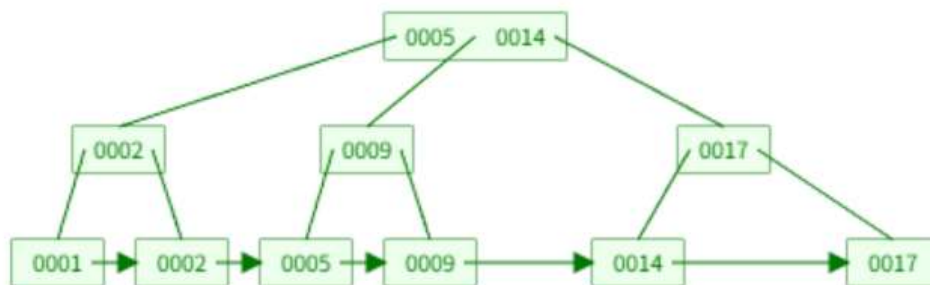
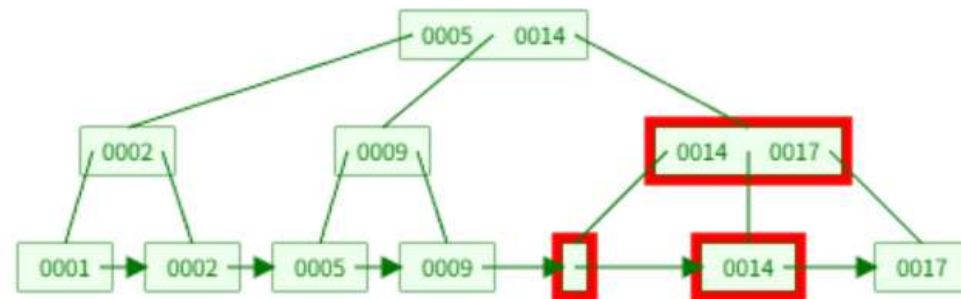
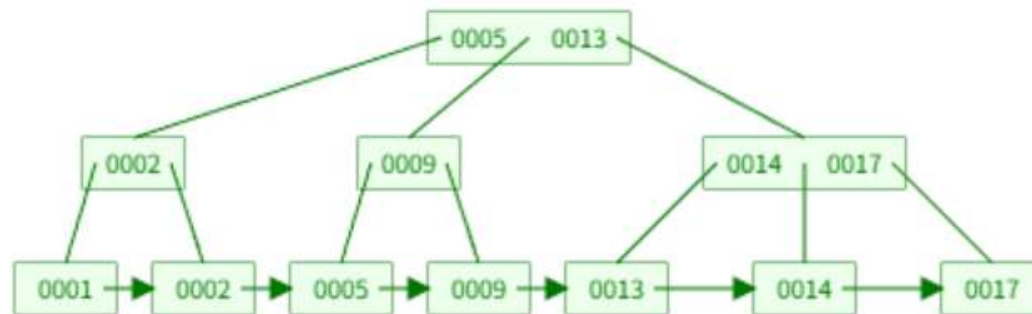
5.B+树的删除

- 情况二：叶子节点删除后需要进行偷取
- 如果左右兄弟节点的键数量大于最小允许数量，可以偷取一个，结束，然后修改两者之间父节点的键值。



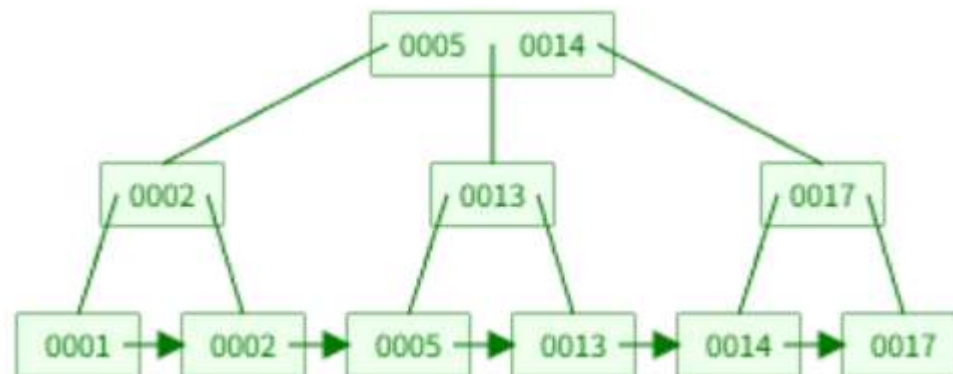
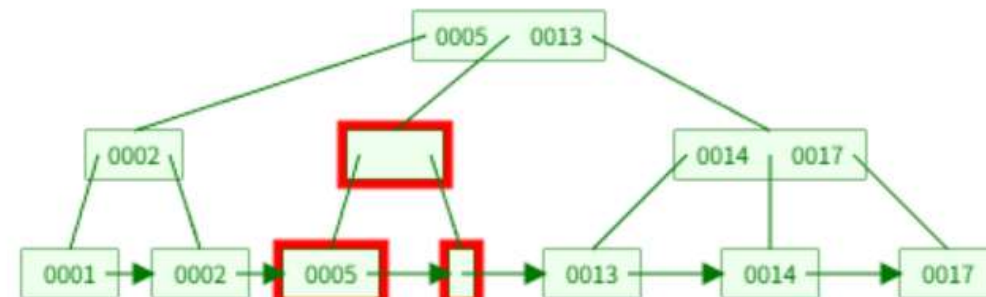
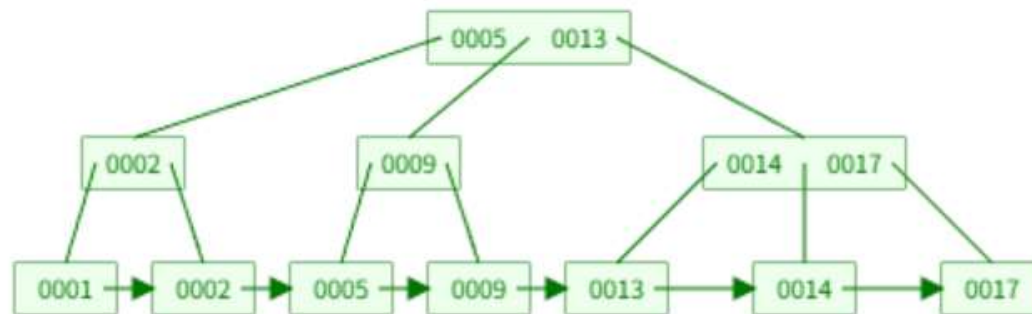
5.B+树的删除

- 情况三：叶子节点删除后需要进行合并
- 如果左右兄弟节点的键数量都恰好等于最小允许数量，选择一个兄弟节点进行合并
- 此时会造成父亲节点（内部节点）的键值减少一个，即内部节点的键值删除操作。



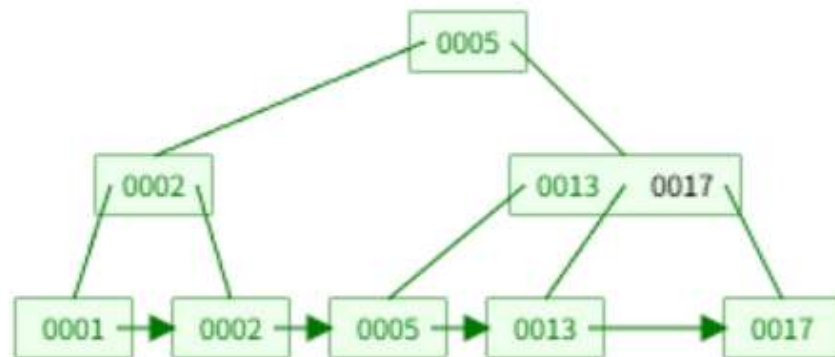
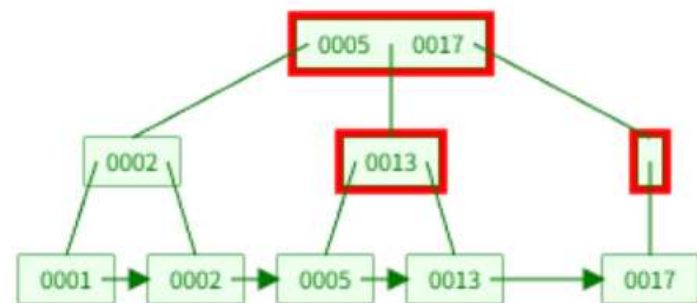
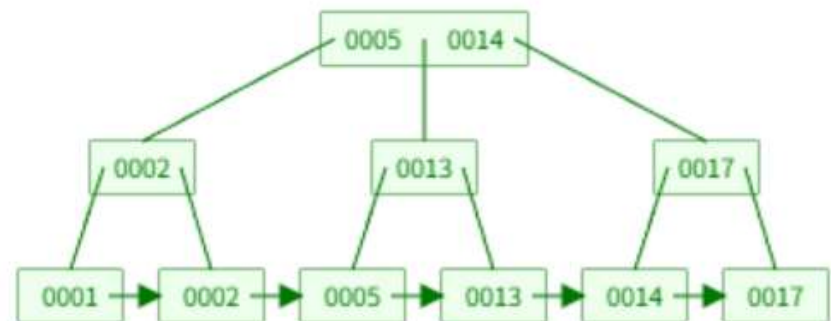
5.B+树的删除

- 情况三：内部节点删除后需要进行偷取
- 如果左右兄弟节点的键数量有大于最小允许数量，选择一个兄弟节点进行偷取。
- 此时的结果是父亲节点的键值下传，偷取的键值上传至父亲节点。



5.B+树的删除

- 情况四：内部节点删除后需要进行合并
- 如果左右兄弟节点的键数量都等于最小允许数量，选择一个兄弟节点进行合并
- 此时的结果是父亲节点的键值下传。



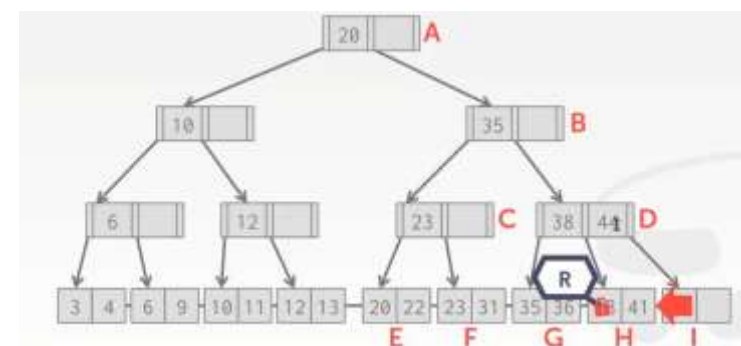
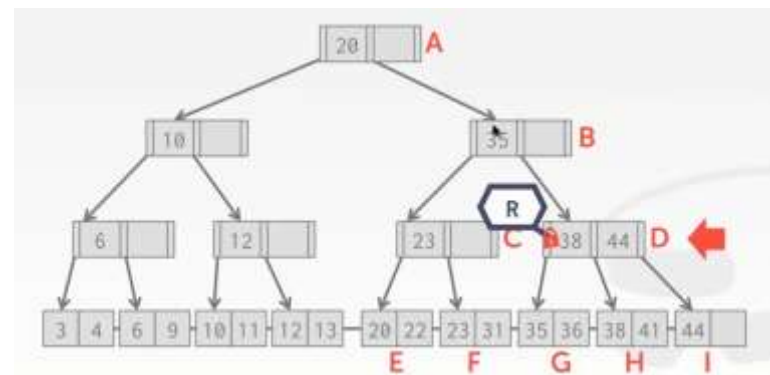
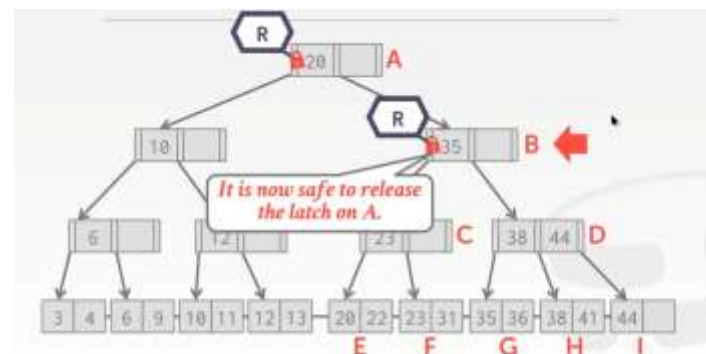
5.B+树的删除

- **情况五：根删除后需要进行合并**
- 如果剩余的儿子为1，则该儿子作为根
- 如果没有儿子，则删除根节点

多线程B+树-Crabbng Latch

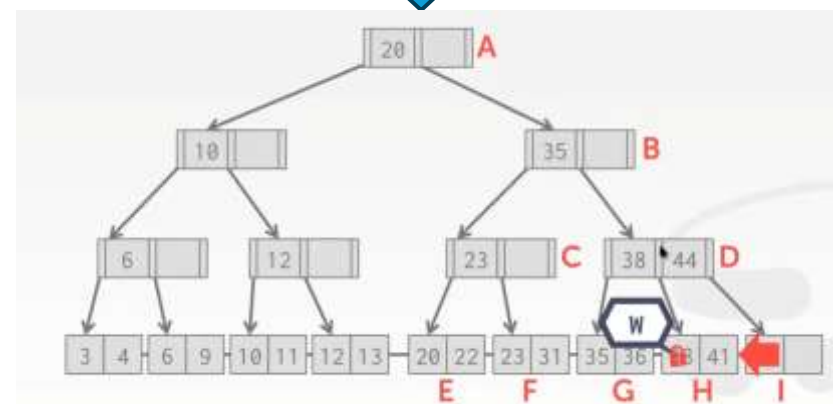
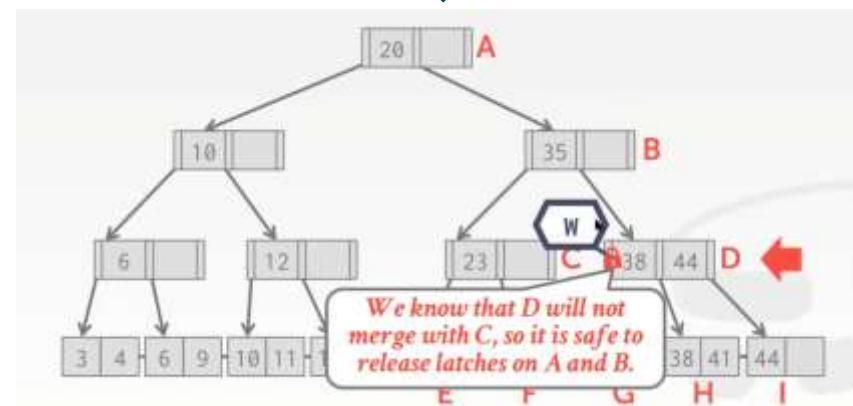
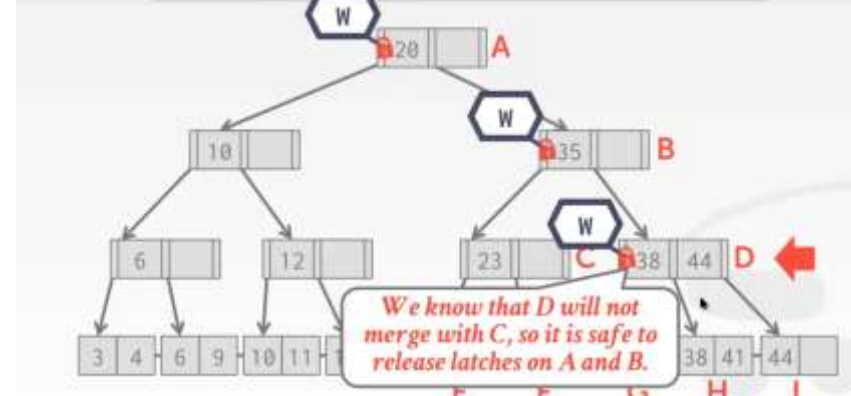
1. 查询操作的上锁流程

- 在 B+ 树上进行读操作时，我们从根节点开始向下搜寻。首先对扫描到的节点加读锁，然后在向下一层时，直接释放父节点的锁
- 这里能直接释放的原因是整个操作是只读的，不会变更 B+ 树节点的状态。



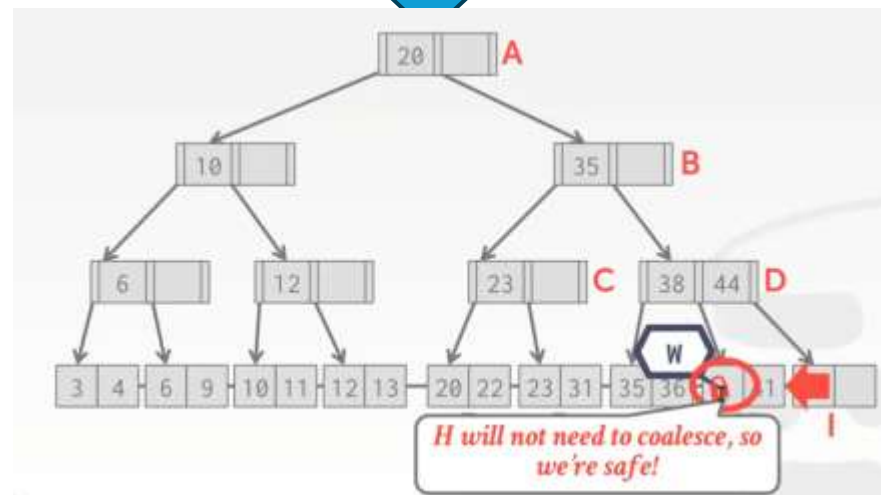
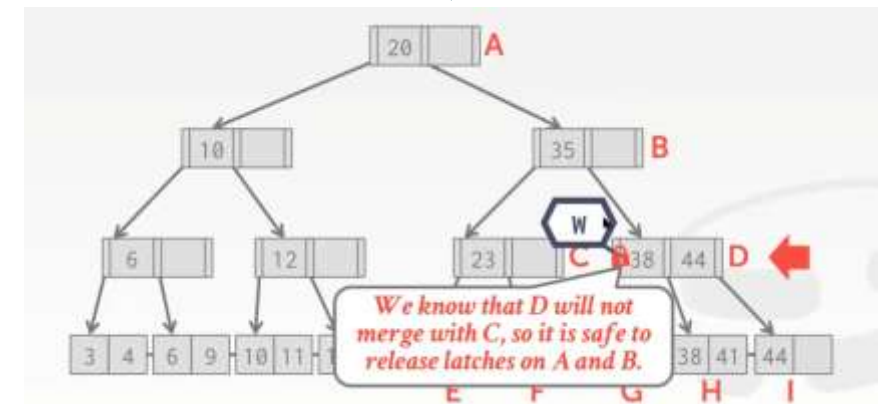
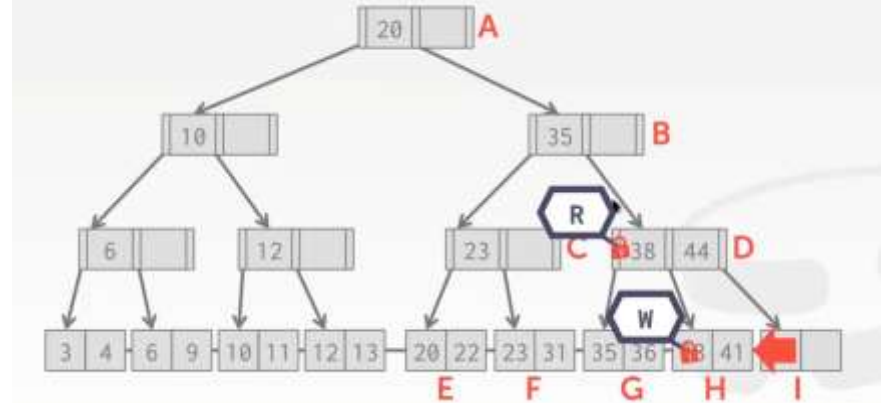
2.删除和插入的上锁流程

- 当需要对 B+ 树进行更新（写入/删除）时，同样从根节点开始搜寻。在经过的节点上加写锁，并进一步判断，如果当前节点被认定为“安全”，那么释放所有父级节点的锁。



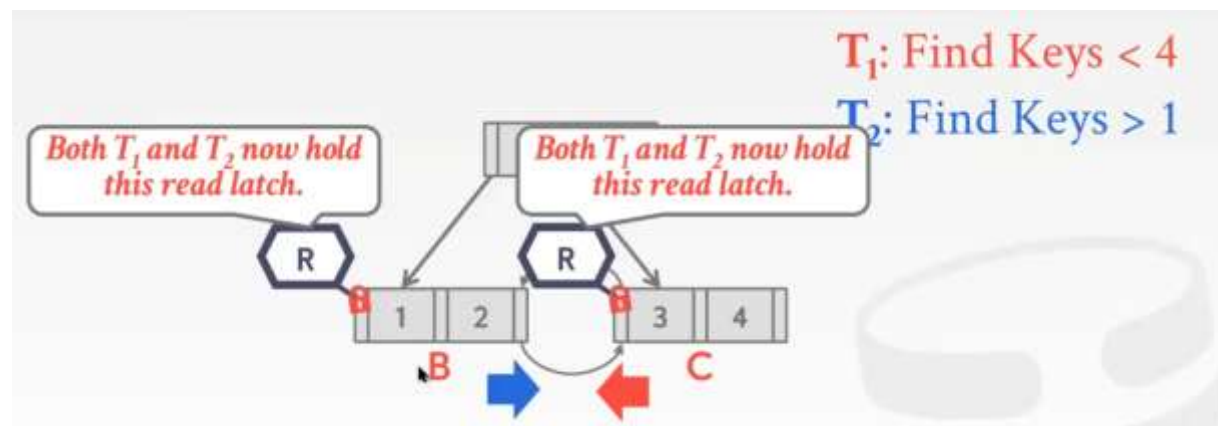
3.存在的问题

- 一、由于所有线程都从根节点开始上锁，如果根节点的锁迟迟没有释放，其他线程就会一直等待，造成效率低下。
- 引入乐观优化。假设修改操作不会引起树的形态发生变化，对沿途的节点上读锁（逻辑与查询操作一直），这样可以多个线程同时进行。如果叶子节点的操作会引起树的形态发生变化，再重新跑一遍，这次沿途上写锁，与之前的删除修改逻辑相同



3.存在的问题

- 二、从上而下的锁操作已经处理好了，但是还有遍历叶子节点的操作还没有上锁操作。
- 假如有一个线程T1从左到右修改叶子节点，有一个线程T2从右到左修改叶子节点，两者互相持有对方的锁，就会造成死锁
- 最简单的解决方式是只允许从左到右进行遍历叶子。



- 节点内二分查找指定键值

```
1 INDEX_TEMPLATE_ARGUMENTS
2 auto B_PLUS_TREE_INTERNAL_PAGE_TYPE::FindKeyPosition(const KeyType &key, KeyComparator &comparator) -> int {
3     int low = 1, high = GetSize() - 1;
4     while(low < high) {
5         int mid = (low + high) / 2;
6         if(comparator(KeyAt(mid), key) >= 0) {
7             high = mid;
8         }
9         else {
10             low = mid + 1;
11         }
12     }
13     if(comparator(KeyAt(low), key) == 0) {
14         return low;
15     }
16     return -1;
17 }
```

- 节点内插入指定键值

```
1 INDEX_TEMPLATE_ARGUMENTS
2 auto B_PLUS_TREE_INTERNAL_PAGE_TYPE::Insert(const KeyType &key, const ValueType &value, KeyComparator &comparator)
3     -> bool {
4     int low = 1, high = GetSize() - 1;
5     while(low < high) {
6         int mid = (low + high) / 2;
7         auto comp_res = comparator(KeyAt(mid), key);
8         if(comp_res >= 0) {
9             high = mid;
10        }
11        else {
12            low = mid + 1;
13        }
14    }
15    if(comparator(KeyAt(low), key) == 0) {
16        return false;
17    }
18    int insert_idx = GetSize();
19    if(comparator(KeyAt(low), key) > 0) {
20        insert_idx = low;
21    }
22
23    ExcavateIndex(insert_idx);
24    SetKeyAt(insert_idx, key);
25    SetValueAt(insert_idx, value);
26    IncreaseSize(1);
27    return true;
28 }
```

- 节点内删除指定键值

```
1 INDEX_TEMPLATE_ARGUMENTS
2 auto B_PLUS_TREE_INTERNAL_PAGE_TYPE::RemoveKey(const KeyType &key, KeyComparator &comparator) -> bool {
3     auto to_delete_idx = FindKeyPosition(key, comparator);
4     if (to_delete_idx == -1) {
5         return false;
6     }
7     FillIndex(to_delete_idx + 1);
8     DecreaseSize(1);
9     return true;
10 }
```


- 节点合并偷取的辅助函数

```
1 INDEX_TEMPLATE_ARGUMENTS
2 void B_PLUS_TREE_INTERNAL_PAGE_TYPE::MoveAllTo(BPlusTreeInternalPage *recipient) {
3     auto recipient_size = recipient->GetSize();
4     auto size = GetSize();
5     std::copy(&array_[0], &array_[size], &recipient->array_[recipient_size]);
6     SetSize(0);
7     recipient->IncreaseSize(size);
8 }
9
10 INDEX_TEMPLATE_ARGUMENTS
11 void B_PLUS_TREE_INTERNAL_PAGE_TYPE::MoveLatterHalfWithOneExtraTo(BPlusTreeInternalPage *recipient, const KeyType &key,
12                                                                    const ValueType &value, KeyComparator &comparator) {
13     BUSTUB_ASSERT(GetSize() == GetMaxSize(), "MoveLatterHalfWithOneExtraTo(): Assert GetSize() == GetMaxSize()");
14     auto total_size = GetSize() + 1;
15     Insert(key, value, comparator);
16     auto remain_size = total_size / 2 + (total_size % 2 != 0);
17     auto move_size = total_size - remain_size;
18     std::copy(&array_[remain_size], &array_[total_size], recipient->array_);
19     SetSize(remain_size);
20     recipient->SetSize(move_size);
21 }
22
23 INDEX_TEMPLATE_ARGUMENTS
24 void B_PLUS_TREE_INTERNAL_PAGE_TYPE::MoveFirstToEndOf(BPlusTreeInternalPage *recipient) {
25     auto recipient_size = recipient->GetSize();
26     recipient->SetKeyAt(recipient_size, KeyAt(0));
27     recipient->SetValueAt(recipient_size, ValueAt(0));
28     recipient->IncreaseSize(1);
29     FillIndex(1);
30     DecreaseSize(1);
31 }
32
33 INDEX_TEMPLATE_ARGUMENTS
34 void B_PLUS_TREE_INTERNAL_PAGE_TYPE::MoveLastToFrontOf(BPlusTreeInternalPage *recipient) {
35     auto size = GetSize();
36     recipient->ExcavateIndex(0);
37     recipient->SetKeyAt(0, KeyAt(size - 1));
38     recipient->SetValueAt(0, ValueAt(size - 1));
39     recipient->IncreaseSize(1);
40     DecreaseSize(1);
41 }
```

- 查找函数

```
1 INDEX_TEMPLATE_ARGUMENTS
2 auto BPLUSTREE_TYPE::GetValue(const KeyType &key, std::vector<ValueType> *result, Transaction *transaction) -> bool {
3     bool dummy_used = false;
4     if (transaction == nullptr) {
5         transaction = new Transaction(0);
6         dummy_used = true;
7     }
8     LatchRootPageId(transaction, LatchMode::READ);
9     if (IsEmpty()) {
10         ReleaseAllLatches(transaction, LatchMode::READ);
11         if (dummy_used) {
12             delete transaction;
13         }
14         return false;
15     }
16     auto [raw_leaf_page, leaf_page] = FindLeafPage(key, transaction, LatchMode::READ);
17     // binary search
18     auto found = false;
19     int low = 0, high = leaf_page->GetSize() - 1;
20     while(low < high) {
21         int mid = (low + high) / 2;
22         auto com_res = comparator_(leaf_page->KeyAt(mid), key);
23         if(com_res >= 0) {
24             high = mid;
25         }
26         else {
27             low = mid + 1;
28         }
29     }
30     if(comparator_(leaf_page->KeyAt(low), key) == 0) {
31         found = true;
32         result->push_back(leaf_page->ValueAt(low));
33     }
34     // clear up all the latches held in this transaction
35     ReleaseAllLatches(transaction, LatchMode::READ);
36     if (dummy_used) {
37         delete transaction;
38     }
39     return found;
40 }
```

查找函数

```
1 INDEX_TEMPLATE_ARGUMENTS
2 auto BPLUS_TREE_TYPE::FindLeafPage(const KeyType &key, Transaction *transaction, LatchMode mode)
3     -> std::pair<Page *, BPlusTreeLeafPage<KeyType, RID, KeyComparator> *> {
4     BUSTUB_ASSERT(root_page_id != INVALID_PAGE_ID, "root_page_id != INVALID_PAGE_ID");
5     if (transaction != nullptr) {
6         // root id latch should already be held in this transaction
7         BUSTUB_ASSERT(transaction->GetPageSet()->front() == nullptr, "transaction->GetPageSet()->front() == nullptr");
8     }
9     auto [raw_curr_page, curr_page] = FetchBPlusTreePage(root_page_id);
10    decltype(raw_curr_page) raw_next_page = nullptr;
11    decltype(curr_page) next_page = nullptr;
12
13    // concurrency support
14    if (transaction != nullptr) {
15        if (mode == LatchMode::READ) {
16            raw_curr_page->RLatch();
17        } else if (mode == LatchMode::OPTIMIZE) {
18            if (curr_page->IsLeafPage()) {
19                raw_curr_page->WLatch();
20            } else {
21                raw_curr_page->RLatch();
22            }
23        } else {
24            raw_curr_page->WLatch();
25        }
26        if (IsSafePage(curr_page, mode)) {
27            ReleaseAlllatches(transaction, mode);
28        }
29        transaction->AddIntoPageSet(raw_curr_page);
30    }
31    while (!curr_page->IsLeafPage()) {
32        auto curr_internal_page = ReinterpretAsInternalPage(curr_page);
33        page_id_t jump_pid = curr_internal_page->SearchPage(key, comparator);
34        BUSTUB_ASSERT(jump_pid != INVALID_PAGE_ID, "jump_pid != INVALID_PAGE_ID");
35        auto next_pair = FetchBPlusTreePage(jump_pid);
36        raw_next_page = next_pair.first;
37        next_page = next_pair.second;
38        // 从下到上依次上锁
39        if (transaction != nullptr) {
40            if (mode == LatchMode::READ) {
41                raw_next_page->RLatch();
42            } else if (mode == LatchMode::OPTIMIZE) {
43                if (next_page->IsLeafPage()) {
44                    raw_next_page->WLatch();
45                } else {
46                    raw_next_page->RLatch();
47                }
48            } else {
49                raw_next_page->WLatch();
50            }
51            if (IsSafePage(next_page, mode)) {
52                ReleaseAlllatches(transaction, mode);
53            }
54            transaction->AddIntoPageSet(raw_next_page);
55        } else {
56            buffer_pool_manager->UnpinPage(curr_page->GetPageId(), false);
57        }
58        raw_curr_page = raw_next_page;
59        curr_page = next_page;
60    }
61    BUSTUB_ASSERT(curr_page->IsLeafPage(), "curr_page->IsLeafPage()");
62    return {raw_curr_page, ReinterpretAsLeafPage(curr_page)};
63 }
```

- 插入函数

```
1 INDEX_TEMPLATE_ARGUMENTS
2 auto BPLUSTREE_TYPE::Insert(const KeyType &key, const ValueType &value, Transaction *transaction) -> bool {
3     bool dummy_used = false;
4     if (transaction == nullptr) {
5         transaction = new Transaction(0);
6         dummy_used = true;
7     }
8     bool success = InsertHelper(key, value, transaction, LatchMode::OPTIMIZE);
9     if (dummy_used) {
10         delete transaction;
11     }
12     return success;
13 }
```

- 删除函数

```
1  INDEX_TEMPLATE_ARGUMENTS
2  void BPLUSTREE_TYPE::Remove(const KeyType &key, Transaction *transaction) {
3      bool dummy_used = false;
4      if (transaction == nullptr) {
5          transaction = new Transaction(0);
6          dummy_used = true;
7      }
8      RemoveHelper(key, transaction, LatchMode::OPTIMIZE);
9      if (dummy_used) {
10         delete transaction;
11     }
12 }
13
```


- 删除函数

```
1 INDEX_TEMPLATE_ARGUMENTS
2 void BPLUSTREE_TYPE::RemoveHelper(const KeyType &key, Transaction *transaction, LatchMode mode) {
3     int dirty_height = 0;
4     LatchRootPageId(transaction, mode);
5     if (IsEmpty()) {
6         ReleaseAllLatches(transaction, mode, dirty_height);
7         return;
8     }
9
10    auto [raw_leaf_page, leaf_page] = FindLeafPage(key, transaction, mode);
11    if ((leaf_page->GetSize() - 1) < leaf_page->GetMinSize() && mode == LatchMode::OPTIMIZE) {
12        // OPTIMIZE mode fail, page should be held in latch before leaf_page
13        auto is_root = leaf_page->IsRootPage();
14        auto is_leaf = leaf_page->IsLeafPage();
15        auto is_internal = leaf_page->IsInternalPage();
16        // root page should be treatment special
17        auto fail_condition1 = !is_root;
18        auto fail_condition2 = is_root && is_leaf && (leaf_page->GetSize() - 1) == 0;
19        auto fail_condition3 = is_root && is_internal && (leaf_page->GetSize() - 1) == 1;
20        if (fail_condition1 || fail_condition2 || fail_condition3) {
21            ReleaseAllLatches(transaction, mode);
22            // 释放所有锁后，从上到下依次重新上写锁
23            return RemoveHelper(key, transaction, LatchMode::DELETE);
24        }
25    }
26    RemoveEntry(leaf_page, key, dirty_height);
27    ReleaseAllLatches(transaction, mode, dirty_height);
28 }
```


- 偷取操作

```
1 INDEX_TEMPLATE_ARGUMENTS
2 void BPLUSTREE_TYPE::Redistribute(BPlusTreePage *base_page, BPlusTreePage *sibling_page,
3                                   BPlusTreeInternalPage<KeyType, page_id_t, KeyComparator> *parent_page, int base_index,
4                                   bool sibling_on_left) {
5     if (base_page->IsLeafPage()) {
6         auto base_leaf = ReinterpretAsLeafPage(base_page);
7         auto sibling_leaf = ReinterpretAsLeafPage(sibling_page);
8         if (sibling_on_left) {
9             // sibling_leaf -> base
10            sibling_leaf->MoveLastToFrontOf(base_leaf);
11            parent_page->SetKeyAt(base_index, base_leaf->KeyAt(0));
12        } else {
13            // base <- sibling_leaf
14            sibling_leaf->MoveFirstToEndOf(base_leaf);
15            parent_page->SetKeyAt(base_index + 1, sibling_leaf->KeyAt(0));
16        }
17    } else {
18        auto base_internal = ReinterpretAsInternalPage(base_page);
19        auto sibling_internal = ReinterpretAsInternalPage(sibling_page);
20        if (sibling_on_left) {
21            // sibling_internal -> base
22            sibling_internal->MoveLastToFrontOf(base_internal);
23            RefreshParentPointer(base_internal, 0);
24            parent_page->SetKeyAt(base_index, base_internal->KeyAt(0));
25        } else {
26            // base <- sibling_internal
27            sibling_internal->MoveFirstToEndOf(base_internal);
28            RefreshParentPointer(base_internal, base_internal->GetSize() - 1);
29            // 还需要修改偷取过来的key为父节点中的key
30            base_internal->SetKeyAt(base_internal->GetSize() - 1, parent_page->KeyAt(base_index + 1));
31            parent_page->SetKeyAt(base_index + 1, sibling_internal->KeyAt(0));
32        }
33    }
34 }
```

- 合并操作

```
1 INDEX_TEMPLATE_ARGUMENTS
2 void BPLUS_TREE_TYPE::Merge(BPlusTreePage *base_page, BPlusTreePage *sibling_page,
3                             BPlusTreeInternalPage<KeyType, page_id_t, KeyComparator> *parent_page, int base_index,
4                             bool sibling_on_left, int &dirty_height) {
5     if (base_page->IsLeafPage()) {
6         auto base_leaf = ReinterpretAsLeafPage(base_page);
7         auto sibling_leaf = ReinterpretAsLeafPage(sibling_page);
8         if (sibling_on_left) {
9             // <----.merge base to left sibling leaf
10            auto key_in_between = parent_page->KeyAt(base_index);
11            base_leaf->MoveAllTo(sibling_leaf);
12            // set next_page_id
13            sibling_leaf->SetNextPageId(base_leaf->GetNextPageId());
14            // mark off the link
15            base_leaf->SetParentPageId(INVALID_PAGE_ID);
16            // 将不满的节点合并到了左边兄弟节点后，需要在父节点中删除不满节点的key，递归的一个过程
17            RemoveEntry(parent_page, key_in_between, dirty_height);
18        } else {
19            // <----.merge right sibling leaf to base
20            auto key_in_between = parent_page->KeyAt(base_index + 1);
21            sibling_leaf->MoveAllTo(base_leaf);
22            // set next_page_id
23            base_leaf->SetNextPageId(sibling_leaf->GetNextPageId());
24            // mark off the link
25            sibling_leaf->SetParentPageId(INVALID_PAGE_ID);
26            //
27            RemoveEntry(parent_page, key_in_between, dirty_height);
28        }
29    } else {
30        auto base_internal = ReinterpretAsInternalPage(base_page);
31        auto sibling_internal = ReinterpretAsInternalPage(sibling_page);
32        if (sibling_on_left) {
33            // <----.merge base to left sibling internal
34            auto key_in_between = parent_page->KeyAt(base_index);
35            auto sibling_old_size = sibling_internal->GetSize();
36            base_internal->MoveAllTo(sibling_internal);
37            // set parent id
38            RefreshAllParentPointer(sibling_internal);
39            sibling_internal->SetKeyAt(sibling_old_size, key_in_between);
40            // mark off the link
41            base_internal->SetParentPageId(INVALID_PAGE_ID);
42            RemoveEntry(parent_page, key_in_between, dirty_height);
43        } else {
44            // <----.merge right sibling internal to base
45            auto key_in_between = parent_page->KeyAt(base_index + 1);
46            auto base_old_size = base_internal->GetSize();
47            sibling_internal->MoveAllTo(base_internal);
48            // set parent id
49            RefreshAllParentPointer(base_internal);
50            base_internal->SetKeyAt(base_old_size, key_in_between);
51            // mark off the link
52            sibling_internal->SetParentPageId(INVALID_PAGE_ID);
53            RemoveEntry(parent_page, key_in_between, dirty_height);
54        }
55    }
56 }
```

```

panan@lancher:~/Desktop/bustub_initial/build$ ./test/b_plus_tree_delete_test
Running main() from gmock_main.cc
[=====] Running 2 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 2 tests from BPlusTreeTests
[ RUN     ] BPlusTreeTests.DeleteTest1
[         OK ] BPlusTreeTests.DeleteTest1 (5 ms)
[ RUN     ] BPlusTreeTests.DeleteTest2
[         OK ] BPlusTreeTests.DeleteTest2 (3 ms)
[-----] 2 tests from BPlusTreeTests (9 ms total)

[-----] Global test environment tear-down
[=====] 2 tests from 1 test suite ran. (10 ms total)
[ PASSED ] 2 tests.
panan@lancher:~/Desktop/bustub_initial/build$

```

```

[100%] Built target b_plus_tree_concurrent_test
panan@lancher:~/Desktop/bustub_initial/build$ ./test/b_plus_tree_concurrent_test
Running main() from gmock_main.cc
[=====] Running 5 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 5 tests from BPlusTreeConcurrentTest
[ RUN     ] BPlusTreeConcurrentTest.InsertTest1
[         OK ] BPlusTreeConcurrentTest.InsertTest1 (211 ns)
[ RUN     ] BPlusTreeConcurrentTest.InsertTest2
[         OK ] BPlusTreeConcurrentTest.InsertTest2 (109 ns)
[ RUN     ] BPlusTreeConcurrentTest.DeleteTest1
[         OK ] BPlusTreeConcurrentTest.DeleteTest1 (4 ms)
[ RUN     ] BPlusTreeConcurrentTest.DeleteTest2
[         OK ] BPlusTreeConcurrentTest.DeleteTest2 (6 ms)
[ RUN     ] BPlusTreeConcurrentTest.MixTest
[         OK ] BPlusTreeConcurrentTest.MixTest (4 ms)
[-----] 5 tests from BPlusTreeConcurrentTest (337 ns total)

[-----] Global test environment tear-down
[=====] 5 tests from 1 test suite ran. (338 ns total)
[ PASSED ] 5 tests.
panan@lancher:~/Desktop/bustub_initial/build$

```

```

[100%] Built target b_plus_tree_insert_test
panan@lancher:~/Desktop/bustub_initial/build$ ./test/b_plus_tree_insert_test
Running main() from gmock_main.cc
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from BPlusTreeConcurrentTestC1
[ RUN     ] BPlusTreeConcurrentTestC1.InsertTest1
[         OK ] BPlusTreeConcurrentTestC1.InsertTest1 (4 ms)
[ RUN     ] BPlusTreeConcurrentTestC1.InsertTest2
[         OK ] BPlusTreeConcurrentTestC1.InsertTest2 (2 ms)
[ RUN     ] BPlusTreeConcurrentTestC1.ScaleTestC1
[         OK ] BPlusTreeConcurrentTestC1.ScaleTestC1 (3918 ms)
[-----] 3 tests from BPlusTreeConcurrentTestC1 (3926 ms total)

[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (3926 ms total)
[ PASSED ] 3 tests.
panan@lancher:~/Desktop/bustub_initial/build$

```